GRAVITAS PROTOCOL

Institutional Technical Evaluation

Governance-Aware Policy Infrastructure, Verifiable Execution Controls and Atomic Digital-Asset Operations

---

Controlled Sandbox Proposal for Exploratory Discussion with the Central Bank of the Republic of Azerbaijan

INTERNAL TECHNICAL REVIEW — NOT AN INDEPENDENT SECURITY AUDIT

Version 1.0 · 11 November 2026 · English

Prepared by Gravitas Protocol for an exploratory technical discussion. This is not a formal application, legal opinion, endorsement or request for regulatory approval.

Contents

1 Executive summary 1
1.1 Decision requested 1
2 The institutional problem and testable proposition 2
2.1 Why this could be relevant to Azerbaijan 2
3 System architecture 3
3.1 Architecture at a glance 3
3.2 Environments and labels 3
4 Majlis governance layer 4
4.1 Governance boundary 4
4.2 Governance maturity and required controls 4
5 Policy Registry 6
5.1 Policy approval and enforcement 6
5.2 Observed Registry state 6
6 Signed Intent 7
6.1 Intent sequence 7
6.2 Signature validation and limitations 7
7 Teleport V2 7
7.1 Scope of V2 7
7.2 V2 failure and integration boundaries 8
8 Teleport V3 9
8.1 Position migration lifecycle 9
8.2 V3 operational controls 10
8.2.1 Questions before a controlled integration 10
9 Atomicity and failure behavior 11
9.1 One transaction: success versus revert 11
9.2 Atomicity evidence and scope 11
10 Sandbox environments 12
10.1 Environment separation 12
10.2 The local reviewer interface 12
11 Reviewer workflow and testing 13
11.1 Independent reviewer workflow 13
11.2 Test methods and status definitions 13
11.3 Executed test suites 14
12 Evidence and audit model 15
12.1 Evidence architecture 15
12.2 Auditability and reproducibility 15
13 Internal technical security review 16
13.1 Findings requiring action 16
13.2 Production-readiness gaps 16
14 Potential use cases in Azerbaijan 17

14.1 Use cases 1–5 17
14.2 Use cases 6–10 17
15 Regulatory and operating position 17
16 Proposed institutional roadmap 19
16.1 Cautious pathway 19
16.2 Readiness and next milestones 19
17 Contract and repository references 19
18 Test catalogue summary 20
18.1 Policy and governance: T01–T09 20
18.2 Signature and intent: T10–T16 20
18.3 Teleport and execution: T17–T25 20
18.4 Atomicity: T26–T30 20
18.5 API and integration: T31–T36 20
18.6 Detailed case evidence tables 20
18.6.1 Policy and governance: T01–T09 21
18.6.2 Signature and intent: T10–T16 22
18.6.3 Teleport execution: T17–T21 23
18.6.4 Teleport execution: T22–T25 24
18.6.5 Atomicity: T26–T30 25
18.6.6 API and integration: T31–T36 26
18.7 Reproduction commands 26
19 Glossary 26
20 References 27
21 Five founder actions 27

1 Executive summary

Gravitas Protocol is a set of software components intended to make selected digital-asset operations more policy-aware and inspectable. The repository contains an application-level governance workflow, an owner-controlled policy Registry, two same-chain liquidity-migration contracts, a TypeScript SDK and a fixture-backed mock API. This report describes what is present in source and tests, what was observed through read-only test-network calls, and what remains a proposal or a material gap.

The current evidence supports a controlled technical discussion, not a production deployment or formal regulatory application. The local Foundry suite uses mock tokens, pools and managers. The test-network review confirmed code and selected getters for documented contracts at one Arbitrum Sepolia block through a third-party RPC provider. It did not establish byte-for-byte correspondence between deployed bytecode and the reviewed source commit, second-provider agreement, or live DEX integration.

The most important architectural boundary is that Majlis does not submit a signed policy update to the Registry. Majlis can record application-level proposals, reasoning, votes, quorum and lifecycle state. Its parameter hash provides canonicalized data-integrity evidence, not a cryptographic approval. A separate, reviewed authorization and transaction path would be needed before an application decision could change on-chain policy.

The package is also explicit about identity and eligibility limits. The Registry's allow-list flags are not KYC, AML/CFT, sanctions screening, accreditation, suitability or beneficial-owner verification. `checkSubscriptionCompliance(subscriber, subscriptionToken)` does not inspect `subscriber`. No production API authentication or hosted compliance service was verified.

The isolated branch adds a loopback-only reviewer interface, a block-pinned read-only Registry snapshot, a restricted local Foundry runner for two fixed scenarios, diagrams, evidence files and a reviewer guide. The example app separates local tests, simulated fixtures and public test-network reads. It has no wallet and no testnet write route.

> **Evaluation status.** Suitable for a preliminary technical discussion. Suitable for exploratory evaluation of local code and test evidence with caveats. Not ready for a formal sandbox application, production institutional integration, custody or handling customer funds. Investor outreach must use the separate investor brief and fill its founder-supplied placeholders before circulation.

1.1 Decision requested

The requested outcome is a short exploratory technical discussion to ask what evidence and eligibility information would be appropriate for a future, narrow evaluation. This document does not ask the Central Bank to approve, license, endorse, sponsor or accept Gravitas into a sandbox. It does not claim that a current application window is open.

Gravitas currently has no Azerbaijan-registered legal entity and no local regulated financial-institution partner. The third-cycle application period was announced as 25 June–25 August 2026 and has closed. The official sources reviewed for this package did not evidence a later open cycle as of 11 November 2026. Any applicant or partner structure requires confirmation from current official rules and Azerbaijani counsel. [1] [2]

2 The institutional problem and testable proposition

Financial institutions need to express control requirements in a form that can be reviewed, applied before an operation, and reconstructed afterward. The proposition worth evaluating is narrow: when a caller uses a component that checks the Policy Registry, an asset, router or executor flag and a pause state can influence whether that operation proceeds. In selected same-chain operations, a single EVM transaction can either complete or revert without partial state from that transaction.

That proposition is not a claim that the Registry intercepts every token transfer. Enforcement occurs only where a contract or caller invokes the relevant Registry check. The Registry does not govern arbitrary contracts, custody flows or external payment systems. The tested examples also do not prove that an institution has adopted the rules, that a policy is legally valid, or that a transaction is Shariah-compliant.

The evidence is useful because a reviewer can inspect the source, tests, an independently reproducible local environment, the public test-network getters, and an explicit failure case. The evidence is limited because most execution tests use deterministic mocks. The package therefore treats reproducibility and visible limitations as more important than marketing claims.

2.1 Why this could be relevant to Azerbaijan

Official CBAR materials describe broad interests in digital finance, data-centric supervisory technology, reporting, operational risk, cybersecurity and resilience. Those priorities offer context for questions about traceability and controlled experimentation; they do not establish demand for Gravitas, a procurement route or a partnership. CBAR has also announced a regulatory sandbox and reported earlier pilot activity. Those facts do not imply approval of this project or a general Islamic-finance regulatory framework. [3] [4] [5]

The appropriate near-term question is whether a narrowly scoped, local-only technical demonstration is useful to review. Any real pilot would need an eligible Azerbaijani applicant structure, a regulated institution if appropriate, activity-specific legal analysis, user protections, AML/CFT controls, data governance, cybersecurity, operational resilience, incident handling, reporting and exit conditions.

3 System architecture

3.1 Architecture at a glance

The following diagram separates implemented components from the missing policy-writing path. The dashed or proposed connections are not working system integrations. Public-chain reads and local EVM transactions use separate environments; the reviewer web app does not submit a testnet transaction.

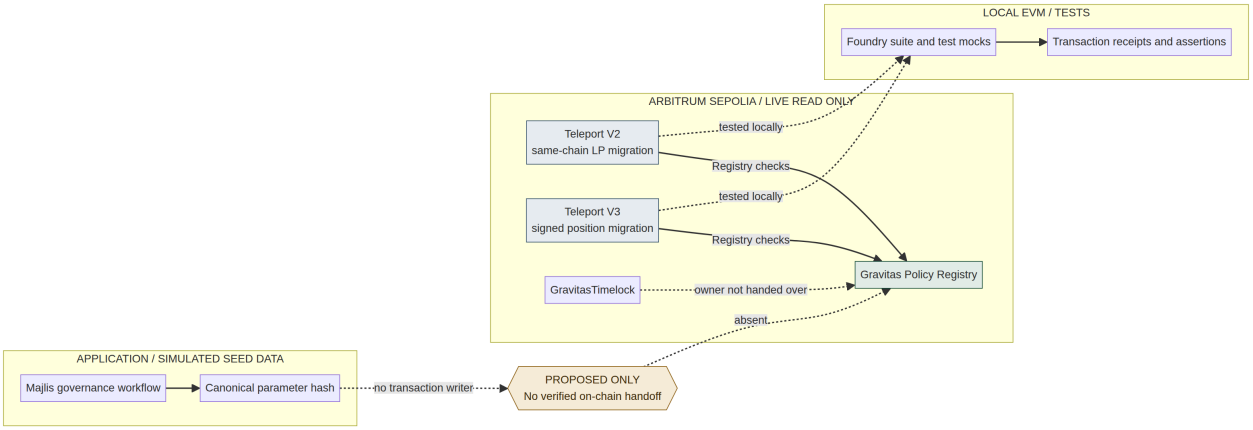


Figure 1: System architecture from repository source; the Majlis-to-Registry transaction handoff is not implemented.

The system has four distinct evidence surfaces. Majlis stores application workflow state. The Registry stores policy flags, pause state and policy history. Teleport V2 and V3 call policy checks as part of specific same-chain operations. The integration kit provides example requests and a local fixture service. These surfaces should not be collapsed into a single “compliance platform” claim.

3.2 Environments and labels

LIVE / DEPLOYED means that a public address had non-empty code or returned a getter at one recorded block through the named RPC provider. It does not mean source-bytecode equality or production suitability. **LOCAL TEST ENVIRONMENT** means Foundry tests with local fixtures. **SIMULATED** means an example result driven by fixture data. **PROPOSED** means a design that is not implemented. **NOT AUDITED** means no independent audit or formal certification has been established.

The reviewer service pins code and getter calls to one snapshot block per refresh. Its HTTP API and upstream RPC errors are described separately: failure to reach the local API is not the same event as a reachable local API reporting an upstream RPC failure. The read-only RPC allow-list accepts only chain, block, code and fixed contract-read methods. Writes are not exposed.

4 Majlis governance layer

4.1 Governance boundary

The Majlis application includes routes and services for proposal lifecycle, deliberation, vote records, role checks, quorum calculation, objections, expiry, document hashing and persistence adapters. The branch documentation corrects an earlier README statement that described Stage One as the only implemented stage. The actual implementation is more than a static mock, but it remains an application workflow. Seeded institutions, boards, members and decisions are demonstration data.

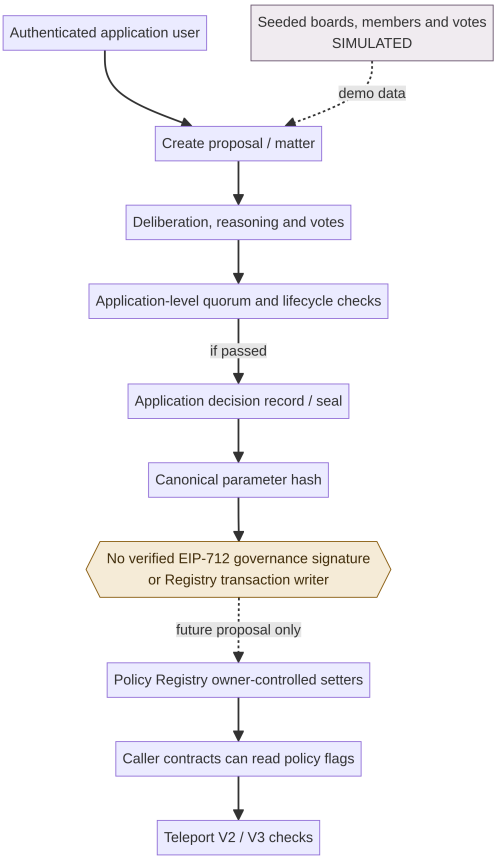


Figure 2: Majlis workflow and the missing chain authorization and Registry transaction handoff.

A reviewer should distinguish three different artifacts: the recorded application decision, the canonicalized parameter hash and an on-chain Registry transaction. The first two exist in the reviewed code. The third is not produced by the current Majlis workflow. A hash is not a signature; an “in force” state in the application is not proof of an on-chain update.

4.2 Governance maturity and required controls

The implementation matrix is: proposal and deliberation workflow — implemented and tested at application level; seeded governance records — mocked/simulated; parameter canonicalization and hash verification — implemented as data-integrity support; cryptographic signing by an authorized institutional signer — not demonstrated; Registry write submission from Majlis — not implemented; independent reconstruction from application record through on-chain event — not end-to-end verified.

Before a real institutional pilot, the team would need to define identity and authority, a cryptographic signing scheme, exact parameter-to-hash binding, separation of proposer and executor, approval thresh-

olds, effective time, timelock and emergency controls, durable evidence retention, and an auditable Registry submission process. Those controls require threat modeling and independent review. A user interface alone cannot supply them.

5 Policy Registry

5.1 Policy approval and enforcement

The Registry stores owner-controlled asset, router and executor flags, a pause state, a policy version and a sequential policy-history hash. Its read functions expose policy state. The contracts that consume the Registry must explicitly call the relevant verification functions. An owner's update changes Registry state; it does not automatically impose a control on every asset or protocol.

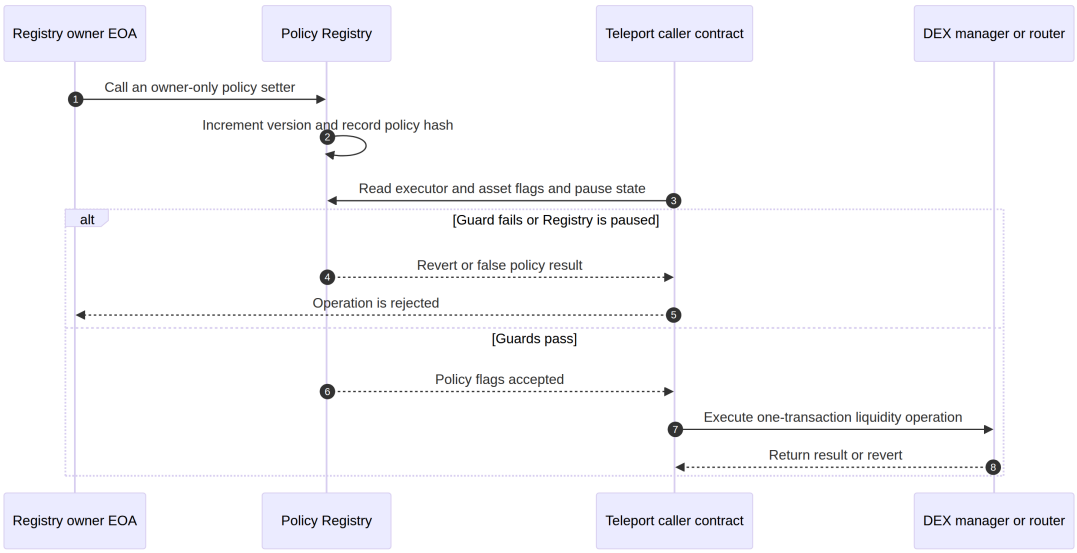


Figure 3: Registry updates and caller-side enforcement; the Registry is not a universal transfer interceptor.

The review found no universal token transfer hook that consults the Registry. When a caller uses a gated verification function and the Registry is paused, that check can revert. A raw mapping getter may still return stored state. An integrator must use the intended gated function for an execution decision rather than infer permission from a raw view.

5.2 Observed Registry state

The Registry address reviewed is `0x6f3bfb896DD9964C9c05dA88692bDf1b1b2C3F23` on Arbitrum Sepolia, chain ID `421614`. At block `328255709`, the recorded snapshot showed owner `0xFba6C221af564Ab90e333AEEf4E427f787911B73`, `pendingOwner()` zero, `paused()` false, current version `1`, and a policy-history hash recorded in `evidence/cbar/arbitrum-sepolia-snapshot-2026-11-11.json`. The snapshot was obtained from the PublicNode endpoint after the official Arbitrum RPC returned HTTP 403 from this sandbox. It is one-provider point-in-time evidence, not an independent deployment audit. Arbiscan exposes source and ABI panels for the addresses. Exact bytecode equality to the reviewed commit and second-provider agreement were not established. [6] [7] [8]

The deployed Timelock has a 300-second minimum delay. The observed owner has proposer role, the zero address has executor role, and the Registry and both engines were not shown as owned by the Timelock. This is not evidence of a completed multisig or timelock handover. The single-step Ownable ownership model in the engines and lack of a separate emergency pauser are material governance limitations.

6 Signed Intent

6.1 Intent sequence

Teleport V3 uses an EIP-712 domain and signed migration parameters. The signed operation binds the position owner, chain and verifying contract context, nonce and deadline, plus operation parameters including token amounts, fee/tick bounds and minimums. The locally generated test signatures are test fixtures; they are not a user authorization and are never exposed as private keys.

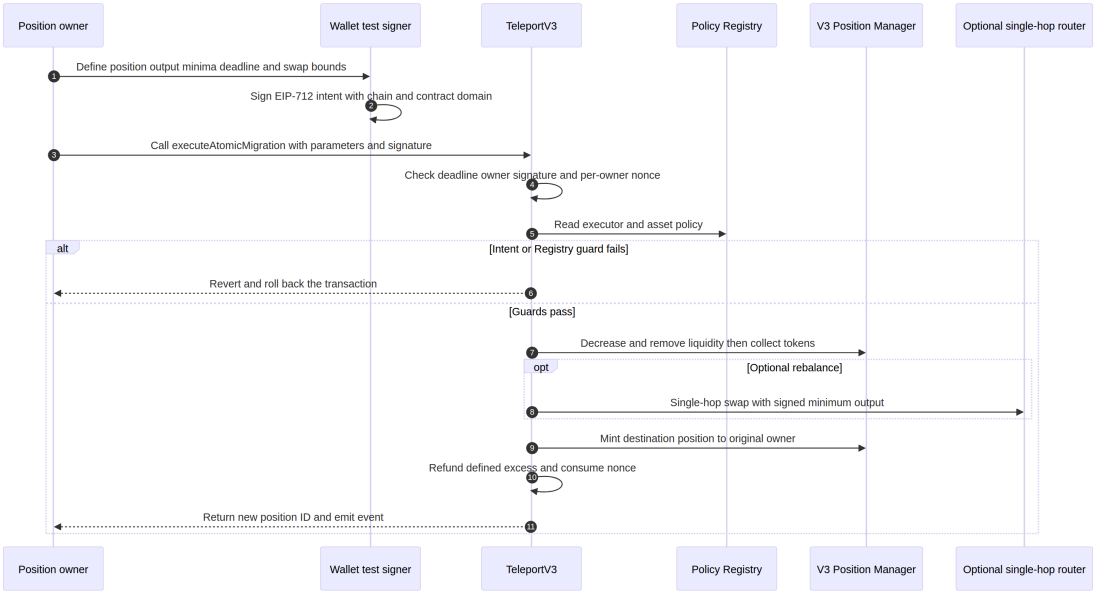


Figure 4: V3 typed-data intent and caller checks; local test signing is distinct from user authorization.

Tests cover valid full flows, parameter modification, swap minimum binding, mint slippage binding, wrong signer, expiry and nonce replay. Dedicated negative tests for a deliberately wrong chain ID and a deliberately wrong verifying-contract address were not found in the executed suite. Domain construction in source is not a substitute for those requested negative tests.

6.2 Signature validation and limitations

The signer is checked against the current position owner. A per-owner nonce prevents reuse of an already executed intent; a deadline rejects expired operations. EIP-712 domain separation binds the signature to the chain and verifying contract. These mechanisms reduce replay and parameter-substitution risks within the tested V3 flow, but they do not establish signer identity, key custody, legal authority or policy approval. Before institutional use, test the exact domain fields against wrong-chain and wrong-contract signatures, key rotation, signer revocation, nonce synchronization and client-side encoding. Validate that the human-readable intent shown to a signer matches the bytes submitted. The current SDK builds calldata and reads/simulates; it does not create wallet signatures or submit transactions.

7 Teleport V2

7.1 Scope of V2

Teleport V2 is a same-chain V2-style liquidity operation. It resolves a pair through a caller-supplied factory, burns provided LP tokens and adds the resulting token pair through an authorized destination

router. It checks relevant Registry executor, router and asset state, applies a per-route cooldown and limits movement relative to total pair liquidity, returns defined dust and includes an owner rescue function.
It is not a cross-chain bridge. The reviewed code does not provide bridge messaging, cross-chain settlement, general multi-hop routing, arbitrary token conversion, a per-user signed intent or a V2-specific pause method. The repository states that no usable V2 DEX route is deployed on Arbitrum Sepolia. Tests use simplified factories, pairs and routers. A production route, liquidity and third-party behavior have not been demonstrated.

7.2 V2 failure and integration boundaries

The principal failure boundaries are route discovery, token and executor policy checks, router authorization, approval/balance constraints, cooldown and movement limits, and external token/router calls. The test suite covers successful mock-backed execution and selected invalid-route and unauthorized-executor cases. Dedicated insufficient allowance and insufficient balance cases remain unexecuted in the independent catalogue.
An institution would need to define permitted tokens, router/factory provenance, custody and signing responsibilities, transaction limits, monitoring and recovery. The deployed V2 address having code is not a substitute for demonstrating a live route or reviewing the exact bytecode.

8 Teleport V3

8.1 Position migration lifecycle

Teleport V3 migrates a Uniswap V3-style liquidity position. It verifies ownership and intent, checks execution conditions, removes liquidity, collects tokens, burns the source position, optionally performs a bounded single-hop rebalance, mints a new position and refunds defined excess balances. The destination position is a new NFT; the original token ID is not preserved.

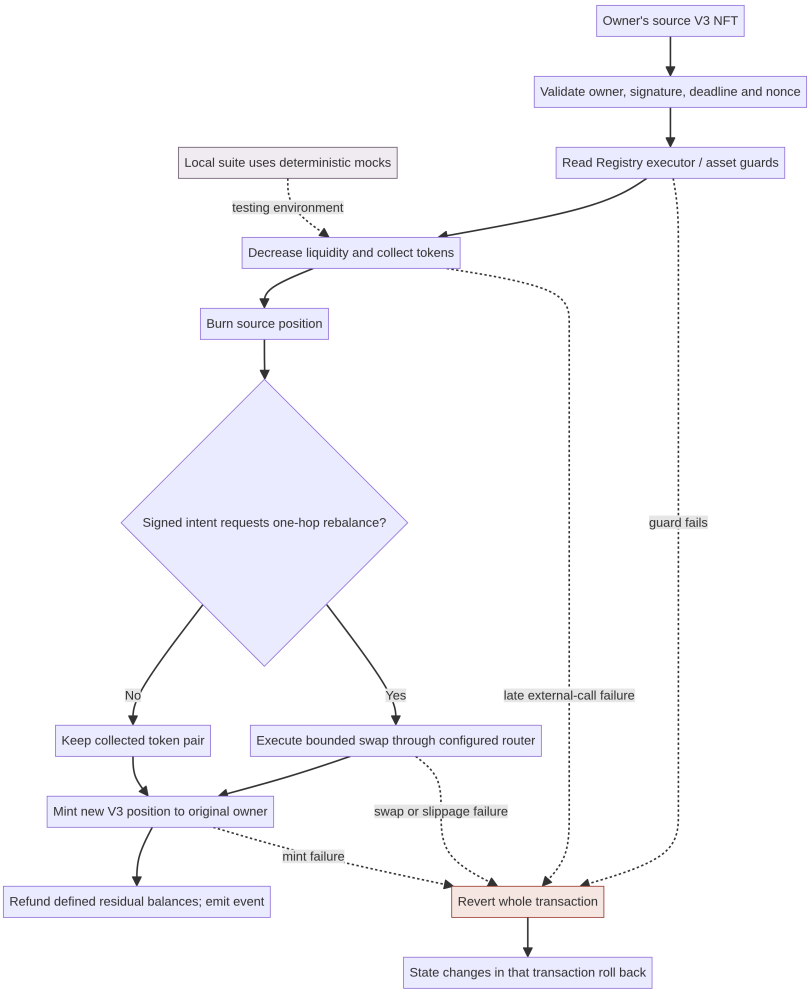


Figure 5: Same-chain V3 position-NFT migration using local mock components in the tested full-flow cases. The current tests exercise no-swap and swap paths with deterministic mocks. They do not show a fork-based or live Uniswap execution. The system is not a central-bank payment network and does not move assets between chains.

8.2 V3 operational controls

The signed migration parameters include the position ID, new fee tier and tick range, mint and decrease minimums, deadline and optional swap settings. The contract checks Registry executor and asset conditions, ownership, signature and nonce. The transaction emits execution evidence and creates a fresh destination position.

The source decrease-side minimums do not require a nonzero floor, so an owner can sign a zero withdrawal minimum. An NFT transferred directly to the contract may be stranded because there is no general rescue path. No separate fast emergency pauser is present. Destination-pool existence and output-below-minimum cases are not directly covered by the requested catalogue. These limitations should be addressed before any institutional or customer-fund use.

8.2.1 Questions before a controlled integration

The following items are review gates, not claims that production integrations are present:

- Confirm token pair, fee tier, tick spacing and pool identity against the position manager and destination route; test invalid or mismatched pools directly.
- Use explicit, non-zero signed minimums where the product requires a lower bound, and test actual below-minimum outputs rather than only verifying that a minimum is signed.
- Review exact-amount approvals, token behavior and external manager/router callbacks; add hostile-token and re-entrancy-focused tests before any customer asset use.
- Specify treatment of a direct ERC-721 transfer to the contract, dust balances, failed external dependencies and any recovery authority without weakening authorization.
- Verify wrong-chain and wrong-verifying-contract signatures, nonce consumption and deadline behavior against the exact client encoding and deployed ABI.

These checks should be performed against pinned dependency versions and the exact deployed implementation in a controlled fork or equivalent reproducible environment. Until then, the current mock-backed results establish only the tested local behavior.

9 Atomicity and failure behavior

9.1 One transaction: success versus revert

The EVM treats a transaction as atomic with respect to state changes in that transaction. When a later external token operation reverts, earlier state changes made within the same transaction are rolled back by the EVM. That property is useful for a multi-step migration, but it does not erase off-chain events, remove market exposure outside the transaction, ensure a retry is safe or prove any religious or legal characteristic.

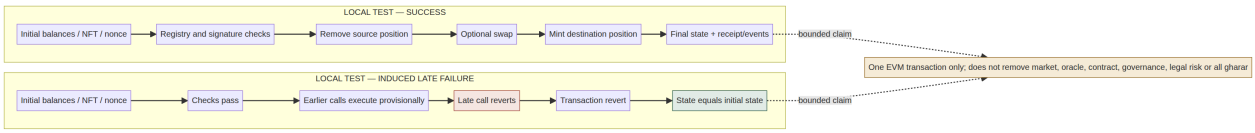


Figure 6: Successful transaction versus induced late revert; rollback is limited to the atomic transaction state.

The branch adds explicit assertions to the late dust-refund failure test. The test snapshots source NFT owner and liquidity, user token balances and the per-owner nonce, then compares those values after the SafeERC20 failure. The targeted test passed 1/1 after the assertions were added.

9.2 Atomicity evidence and scope

The targeted failure was executed in the local EVM with mock tokens and managers. The evidence supports rollback for the state fields asserted in that scenario. It is not a formal invariant over every storage slot, every external dependency or every failure point. Other negative cases—insufficient allowance, insufficient balance, invalid destination pool and a true below-minimum market output—remain unexecuted in the independent catalogue.

Atomicity can reduce partial-execution uncertainty in a single transaction. It does not eliminate smart-contract risk, market risk, oracle risk, liquidity risk, governance risk, key risk, legal risk or operational risk. It does not independently establish Shariah compliance and must not be described as a complete solution to gharar.

10 Sandbox environments

10.1 Environment separation

The reviewer package separates mock mode, local EVM transactions, public test-network reads and test-network writes. Only the first three are available in this branch. Test-network writes are deliberately absent because the package does not establish an authorized signer, supported live route, safe liquidity conditions or a verified exact deployment/source match.

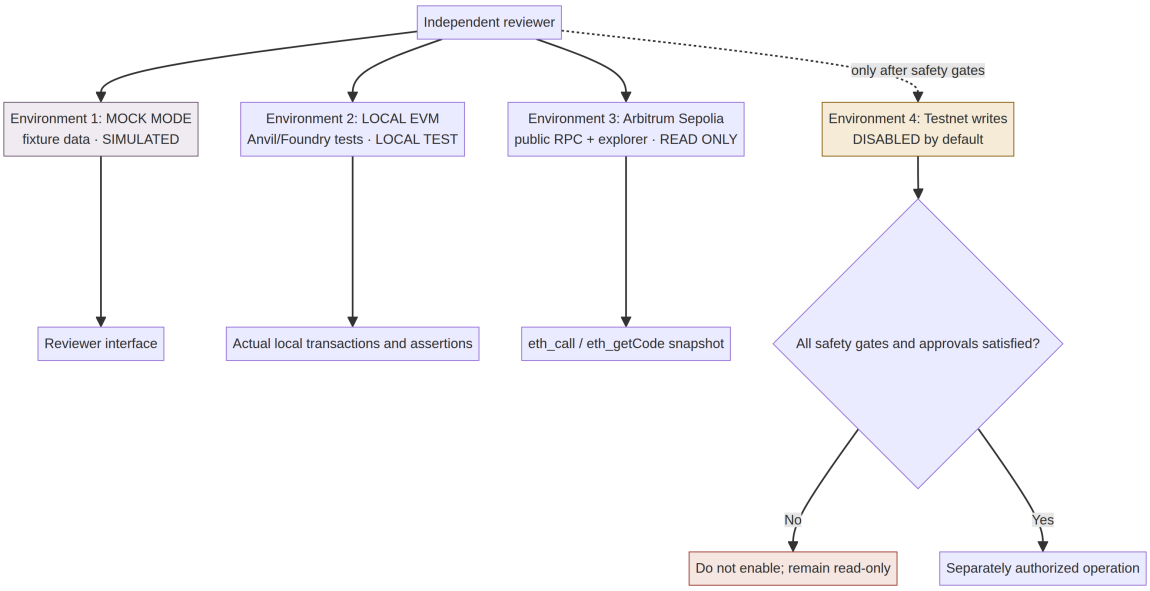


Figure 7: Mock, local EVM and read-only Sepolia modes are separated; testnet writes are not enabled.

The local reviewer service binds to loopback. Its only POST endpoint accepts an exact allow-listed action, requires loopback and same-origin checks, launches fixed Foundry commands with `shell:false`, a minimal environment and no inherited credentials, and serializes runs. It does not accept arbitrary shell commands, signer data, wallet keys or network-write actions.

10.2 The local reviewer interface

The reviewer interface presents overview, architecture, Majlis, Registry, Signed Intent, V2, V3, atomicity lab, test catalogue, evidence and limitations. It makes a block-pinned snapshot from a fixed address and ABI allow-list. Each contract getter and bytecode read uses the same reported block tag. Invalid addresses are rejected before upstream RPC calls; write-like HTTP methods are rejected.

The live-read UI distinguishes a local application/server failure from an upstream RPC error returned through the reachable service. A failed upstream RPC is not a compliance denial, and a reachable API is not proof that an underlying RPC call succeeded. Current results are captured at one provider and should be cross-checked before institutional reliance.

The app's local test buttons run only two fixed Foundry paths. The mock API's six scenarios run separately against local fixture policy data. Green UI indicators are not sufficient evidence by themselves; the reviewer guide provides command-line reproduction and captured output paths.

11 Reviewer workflow and testing

11.1 Independent reviewer workflow

A reviewer should first confirm the branch and source revision, then read the reviewer guide, inspect the relevant Solidity and Majlis code, run the local tests, compare expected and actual outcomes, inspect the read-only snapshot, and finally record any mismatch. The process should not require Gravitas staff to interpret a button result.

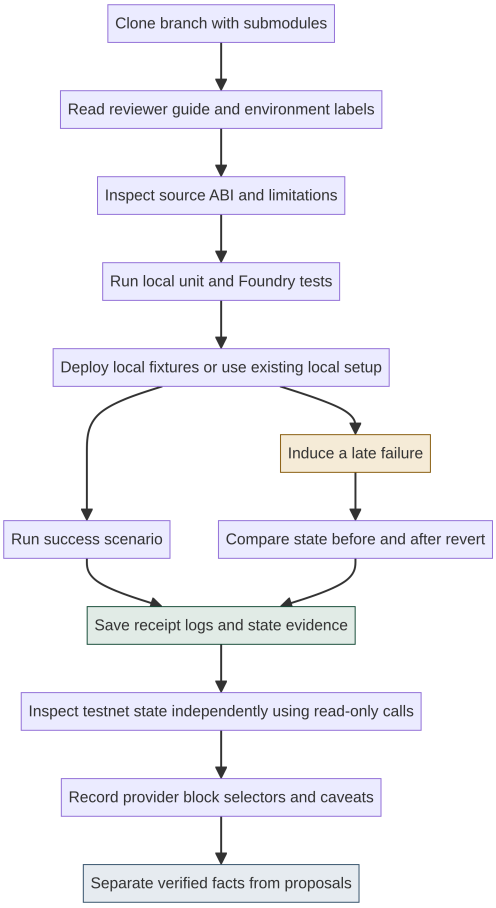


Figure 8: Reproducible reviewer sequence from source inspection through evidence capture.

The guide includes prerequisites, recursive submodule setup, local configuration, commands for Foundry, Majlis, SDK, mock API and reviewer service, success and failure reproduction, log inspection and troubleshooting. It keeps one-provider network evidence separate from local transaction receipts.

11.2 Test methods and status definitions

The test catalogue contains 36 requested cases. A `PASS` is limited to the behavior asserted by an executed test. `PASS, SIMULATED` means the local fixture response matched the sample expectation, not that a live service or chain did. `PASS, LIMITED` means only the named state fields or event replay path were checked. `NOT EXECUTED` means the exact requested scenario has no direct executed evidence.

Current catalogue summary: 22 local contract or Registry event cases are marked PASS or PASS, LIMITED; two mock API cases are PASS, SIMULATED; twelve requested cases remain NOT EXECUTED. No testnet transaction hash exists. Local Foundry tests use freshly deployed fixtures and therefore have no public

transaction hash. The detailed catalogue with preconditions, expected/actual outcomes, commands and evidence paths is `docs/cbar/CBAR_TEST_PLAN.md`.

11.3 Executed test suites

Final isolated-branch verification on 11 November 2026: `forge fmt --check`, `forge build --sizes` and the complete Foundry suite passed; all 86 root tests passed with 0 failures and 0 skips. The compiler's unused-`subscriber` warning remains visible. Majlis `npm ci`, typecheck, tests and build passed (56 test files, 488 tests); existing React test and Vite chunk warnings remain visible. The SDK `npm ci`, tests and build passed (20 tests); its npm audit still reports two high and two moderate development-tool advisories. The reviewer app passed all 4 Node tests and syntax/JSON checks. The local mock integration example returned 6/6 expected fixture decisions, all simulated. Full results are in `docs/cbar/CBAR_TEST_RESULTS.md`; captured raw outputs are in `evidence/cbar/`.

The mock API rejects one malformed request missing required `amount` with HTTP 422. This is limited request-shape validation; the mock does not validate the complete OpenAPI schema or a response through a fail-closed API client. The SDK has no active HTTP compliance-API client. API-unavailable, malformed response and incorrect policy-version cases therefore remain NOT EXECUTED.

12 Evidence and audit model

12.1 Evidence architecture

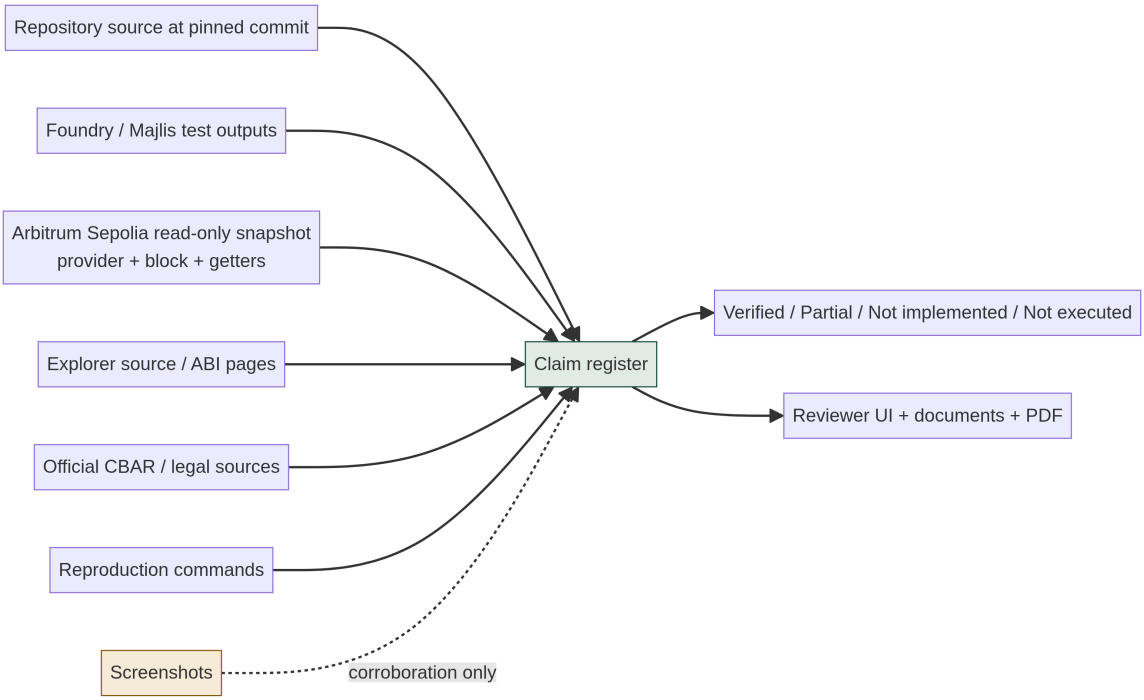


Figure 9: Evidence sources and their boundaries; screenshots supplement but do not replace machine-readable evidence.

The evidence index links each claim to source, test, RPC snapshot, official source or reproducible command. Local logs and public-chain state are not interchangeable. A test output records execution in a local EVM; a public RPC snapshot records selected state at a block; an explorer page exposes source/ABI material; none of these alone proves bytecode equivalence or overall security.

Evidence artifacts include the block-pinned JSON snapshot, local Foundry command output, SDK unit-test/build output, mock API scenario output, mock audit JSONL and dependency audit records. No API key, private key, customer record or real institutional vote is included. Synthetic IDs and seeded demo data are labeled as such.

12.2 Auditability and reproducibility

The Registry policy hash can be reconstructed from ordered update events in the local test suite. The hash demonstrates a commitment to parameter data; it is not an approval signature. A complete governance evidence chain would need to link the authenticated proposal, reasoned deliberation, authorized signer, exact typed data, transaction, Registry event, resulting state and an independently retained record.

The local mock audit log records timestamp, request ID, channel, asset, amount and fixture decision as JSON Lines. The service writes locally and does not query the live Registry. The format is not a tamper-evident production audit service. OpenAPI declares `X-Gravitas-Key`, but the local mock does not enforce it. The service binds to `127.0.0.1` to reduce accidental network exposure; it must not be exposed as a shared service.

13 Internal technical security review

13.1 Findings requiring action

This is an internal technical review, not an independent security audit, penetration test, certification or legal/Shariah opinion. Positive evidence includes owner-gated Registry setters, policy version/hash tests, caller-side Registry checks in V2/V3, EIP-712 binding and replay/expiry tests, SafeERC20 use, and an explicit late-refund rollback comparison.

Material findings remain. Majlis has no verified Registry write path. Live Registry and engines were not shown as Timelock-owned; observed Registry pending owner was zero. The Timelock delay is 300 seconds and the zero address has executor role. The Registry does not inspect subscriber identity. V2 has no standalone pause interface and no documented live route. V3 permits a zero decrease minimum and lacks a general accidental-NFT rescue path. There is no separate fast pauser.

The SDK's paused pair-read path may expose a raw revert instead of a normalized pause error. The local API lacks authentication middleware. The SDK audit reports two high and two moderate advisories in Vitest/Vite-related development dependencies; a compatible patch update attempt failed in npm's resolver. Zod was updated to 4.6.7 and the SDK tests/build passed, but maintainers should review this major dependency change before release. The mock server's compatible dependency updates removed its current npm audit findings.

13.2 Production-readiness gaps

Before handling customer assets, the project needs an independent contract security review; a verified deployment/source match; robust governance, signer, owner and executor architecture; explicit emergency and recovery controls; negative tests for wrong chain and verifying contract; more fault injection; real integration tests against target protocols; production API authentication, authorization, monitoring and response validation; and operational incident, data retention, privacy and exit controls.

No independent audit, regulatory approval, certification, banking partnership, customer deployment, revenue, pilot or investment commitment is claimed. No Azerbaijan legal entity exists. The package is suitable only for controlled code review and exploratory technical discussion. These findings must remain visible in all materials.

14 Potential use cases in Azerbaijan

14.1 Use cases 1–5

Policy-controlled digital-asset operations. A narrow test can show how a caller blocks an operation when a Registry flag or pause check fails. It requires a regulated institution to define the asset set, authorization roles and legal activity. Local test tokens are appropriate now; testnet is read-only.

Traceable approval workflows. Majlis can show proposal, deliberation and vote records with seeded demo data. Production identity, durable records, retention and independent verification remain required. No seeded institution or vote is real.

Machine-readable financial policies. Registry asset, router and executor flags plus version/hash offer a simple machine-readable state model. An institution would need a finite policy schema, provenance, effective dates and change control. Governance-to-Registry signing remains unimplemented.

Institutional governance and auditability. Majlis records and Registry events provide separate evidence. The missing link is authenticated authority from a decision to the exact on-chain update. A local test can compare data and event history but cannot be called a real board approval.

Controlled digital-finance experimentation. Foundry tests permit local transaction success and failure without live funds. A formal sandbox would also need a legally eligible applicant, user safeguards, risk controls, agreement and exit process.

14.2 Use cases 6–10

RegTech infrastructure. Read checks, hashes and a local mock can support a technical discussion. Real institution APIs, authentication, security review, audit retention and operational service controls are absent.

Operational risk reduction. The rollback test demonstrates a narrow single-transaction property. Monitoring, retries, recovery, external dependencies and incident processes remain separate requirements.

Shariah-conscious product governance. Structured parameters and policy versions can support a qualified review process. No Shariah board, ruling, product certification or ongoing qualified governance is represented in this repository.

Financial-institution integration. The SDK reads, simulates and builds calldata; the example REST mock returns fixture decisions. Neither is a live bank integration. Authentication, APIs, SLAs, responsibilities and regulatory classification must be agreed with a licensed institution.

Controlled testing of tokenized operations. V2-style LP and V3 position-NFT migration can be tested with local mocks. It is not a generic tokenized deposit, payment or central-bank money system. Live assets, funds and institutional participants are not used.

These use cases are hypotheses, not existing customer demand or verified market size. A technical demonstration should be selected only after a prospective institution defines the problem and lawful scope.

15 Regulatory and operating position

The CBAR Regulation defines the applicant as an Azerbaijani-registered legal entity seeking to test an innovative product in Azerbaijan. Portal categories and collaboration descriptions do not remove the local legal-entity requirement. Any regulated institution as a possible partner or applicant requires confirmation for the specific activity. A sandbox is a supervised, agreement-based test, not a general license, blanket

waiver, product approval or endorsement. The published third-cycle period ended on 25 August 2026; no later open window was evidenced in the reviewed official sources as of the research cutoff. [9] [10]
Before a future pilot, the team would need written guidance on applicant eligibility and local registration, product classification, user protection, AML/CFT, risk analysis, privacy and data retention, cybersecurity, operational resilience, test limits, reporting, duration and exit. The official regulation states a general maximum test period of 12 months with a possible extension up to six months subject to conditions and CBAR approval; legal counsel should check the authoritative local-language text before external reliance. [9]

16 Proposed institutional roadmap

16.1 Cautious pathway

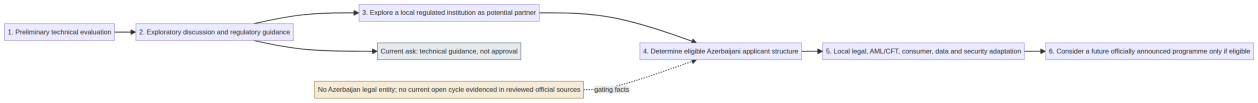


Figure 10: Proposed staged pathway; no open CBAR cycle, local applicant or partner is assumed.

The near-term step is a preliminary technical evaluation using the local reviewer guide and read-only public test-network snapshot. A second step is an exploratory discussion to ask which route, applicant criteria and evidence should be considered. If technically relevant, a local regulated financial institution could be explored as an independent potential partner. Azerbaijani counsel and CBAR would need to clarify the eligible applicant structure and activity classification.

Only after those steps should the team adapt user safeguards, AML/CFT, data, cybersecurity, resilience, risk, reporting and exit controls. A future formal application would depend on eligibility, readiness and a publicly announced window. No application or contact has been made through this work.

16.2 Readiness and next milestones

Preliminary technical discussion: ready to request, with this package and candid caveats. **Exploratory institutional evaluation:** conditionally ready if reviewers accept a local, mock-backed demonstration and one-provider read-only snapshot. **Local bank partnership discussion:** exploratory only; no current partner or institution-specific integration. **Formal sandbox application:** not ready; no Azerbaijan legal entity and no evidenced open window. **Investor outreach:** possible only with the separate investor brief completed with verified team, funding and market data; no CBAR endorsement should be implied.

The next engineering milestones are: independently verify deployed bytecode and state through a second provider; add wrong-chain and wrong-verifying-contract negative tests; design the governance-to-signer-to-Registry path; resolve Timelock, executor and emergency-control design; complete API authentication and response validation; and test real external protocol integrations in a controlled fork environment. Each milestone requires its own threat review and explicit evidence.

17 Contract and repository references

The inspected source baseline is commit `692022e1ca34a280f90a198d2455ac3535b3bce2` on `origin/main`, from which the isolated `cbar-azerbaijan-sandbox` branch was created. This branch changes documentation, reviewer tooling, a local mock API security binding, SDK dependency files and a V3 test assertion. No production Solidity contract implementation was changed in this work.

The key addresses are: Policy Registry `0x6f3bfb896DD9964C9c05dA88692bDf1b1b2C3F23`; Teleport V3 `0x6702C2CE6eD58ca3934eBBd785CaC1De8DCd85B4`; Teleport V2 `0xEDfF3dFdcdd7C04B11d9B614D5e0cd368f1e93c0`; Timelock `0xbFFAd90B2607e3E5926260B640BbcD1E128680Ba`. These are documented Arbitrum Sepolia addresses. Code presence and selected getters are not proof of source equivalence, security or production operation.

Relevant files include `contracts/GravitasPolicyRegistry.sol`, `contracts/TeleportV2.sol`, `contracts/TeleportV3.sol`, `contracts/GravitasTimelock.sol`, `apps/majlis/server/src/routes/governance.ts`, `apps/majlis/server/src/services/lifecycle.ts`, `apps/majlis/server/src/`

`services/hash.ts`, `gravitas-sdk/src/compliance.ts`, `integration-kit/mock-server/server.js` and `test/foundry/TeleportV3FullFlow.t.sol`.

18 Test catalogue summary

The detailed 36-case catalogue is in `docs/cbar/CBAR_TEST_PLAN.md`. It states each test's purpose, preconditions, initial state, input or setup, command, expected result, actual status, source test and evidence. The following summary is not a substitute for those instructions.

18.1 Policy and governance: T01–T09

T01–T07 are marked PASS for the named Registry fixture tests covering accepted and rejected assets, pause, executor authorization, policy version, hash replay and parameter guards. T08, invalid governance authorization, and T09, an incomplete application decision presented as a chain update, are NOT EXECUTED because the integrated authorization path does not exist. Source inspection documents that boundary but is not a passing end-to-end test.

18.2 Signature and intent: T10–T16

T10 and T11 pass in local full-flow and parameter-binding tests. T14 deadline, T15 nonce replay and T16 wrong signer pass in local tests. T12 wrong chain ID and T13 wrong verifying contract are NOT EXECUTED; domain construction in source is not counted as direct negative-test evidence.

18.3 Teleport and execution: T17–T25

T17 valid V2 migration, T18 invalid route, T21 valid V3 migration and T25 unauthorized executor are PASS using local mocks. T19 insufficient allowance, T20 insufficient balance, T22 invalid destination pool, T23 below-minimum output, and T24 invalid NFT ownership are NOT EXECUTED as exact direct cases.

18.4 Atomicity: T26–T30

T26 successful full execution and T27 induced late failure pass in local Foundry tests. T28 compares source owner, liquidity, user balances and nonce before/after the revert; it passes for those fields only. T29 covers named zero-dust/with-dust behaviors. T30 confirms no partial changes for the same tested failure path. No public transaction hash exists for these local calls.

18.5 API and integration: T31–T36

T31 passes as SIMULATED: six fixture scenarios return their expected sample decisions, and a direct valid request returns `ALLOW` from policy set `SANDBOX-2026-07`. T34 passes as SIMULATED and LIMITED: the mock returns HTTP 422 when the required amount is missing. T32 unavailable API, T33 invalid response and T35 incorrect policy version are NOT EXECUTED because no hosted API or active API client is established. T36 passes LIMITED for Registry event replay in a local fixture; it is not hosted API audit-log consistency.

18.6 Detailed case evidence tables

The compact tables below expose the expected outcome and evidence route for every requested case. Full preconditions, initial states, inputs, commands, actual results and artifact paths remain in `docs/cbar/CBAR_TEST_PLAN.md`. No local fixture is represented as a public transaction.

18.6.1 Policy and governance: T01–T09

ID	Scenario and setup	Result and evidence
T01	Approved asset, local Registry test token and authorized executor	PASS — `test_CheckSubscriptionCompliance_Success`; returns the policy version.
T02	Unapproved token presented to the subscription check	PASS — `test_CheckSubscriptionCompliance_RevertOnNonCompliantAsset`; check reverts.
T03	Owner pauses local Registry before authorization check	PASS — `test_Registry_PauseHaltsExecutorAuthorizationSystemWide`; gated check reverts.
T04	Caller lacks Registry executor authorization	PASS — subscription, V2 and V3 unauthorized-executor tests revert.
T05	Owner changes a Registry flag and reads version	PASS — versioning tests confirm a genuine update advances the version.
T06	Replay ordered policy events and compare commitment	PASS — observer replay and policy-hash differentiation tests pass.
T07	Attempt zero-address and non-owner Registry writes	PASS — zero-address and owner-only setter tests revert as expected.
T08	Submit invalid governance authorization to Registry	NOT EXECUTED — no verified Majlis signing or Registry writer path exists.
T09	Distinguish application approval from an on-chain update	NOT EXECUTED — no machine-enforced end-to-end assertion; documented boundary only.

18.6.2 Signature and intent: T10–T16

ID	Scenario and setup	Result and evidence
T10	Owner signs valid V3 migration intent with correct domain, nonce and deadline	PASS — local full-flow tests for no-swap and swap migration.
T11	Change a signed parameter or minimum after signing	PASS — modified-parameter, swap-minimum and mint-minimum binding tests revert.
T12	Sign for a deliberately different chain ID	NOT EXECUTED — no direct wrong-chain negative test was run.
T13	Sign for a deliberately different verifying contract	NOT EXECUTED — no direct wrong-verifying-contract negative test was run.
T14	Submit an intent whose deadline has expired	PASS — `test_V3_RevertOnDeadline`; call reverts.
T15	Replay an already executed owner nonce	PASS — `test_V3_NonceReplayProtection`; second call reverts.
T16	Sign with an account unrelated to the position owner	PASS — `test_V3_WrongSigner`; call reverts.

18.6.3 Teleport execution: T17–T21

ID	Scenario and setup	Result and evidence
T17	V2 migration through local V2 pair, factory and router fixtures	PASS — `test_V2_FullMigrationFlow_MaxMoveBpsEnforcement`; mock DEX only.
T18	V2 factory cannot resolve a valid pair route	PASS — `test_V2_RevertOnPairNotFound`; migration reverts.
T19	User allowance is below the required token or LP amount	NOT EXECUTED — no direct insufficient-allowance case identified.
T20	User balance is below the required token or LP amount	NOT EXECUTED — no direct insufficient-balance case identified.
T21	Valid signed V3 position migration with local pool mocks	PASS — full-flow no-swap and swap tests; new NFT position is minted.

18.6.4 Teleport execution: T22–T25

ID	Scenario and setup	Result and evidence
T22	Request a destination configuration with no valid destination pool	NOT EXECUTED — fee/tick tests are related but not equivalent.
T23	Actual output falls below signed minimum	NOT EXECUTED — binding tests do not simulate a below-minimum market output.
T24	Otherwise valid intent refers to an NFT not owned or approved by signer	NOT EXECUTED — wrong-signer test does not prove this ownership case.
T25	Non-authorized executor calls V2 or V3 migration	PASS — both V2 and V3 unauthorized-executor tests revert.

18.6.5 Atomicity: T26–T30

ID	Scenario and setup	Result and evidence
T26	Complete full V3 migration through all local mock steps	PASS — no-swap and swap full-flow assertions pass.
T27	Make the late dust-refund token transfer fail	PASS — SafeERC20 failure reverts; targeted test passed 1/1.
T28	Compare source NFT owner/ liquidity, user balances and nonce around revert	PASS — asserted state fields equal their initial values after revert.
T29	Exercise zero-dust and defined-dust success cases	PASS — named zero-dust/with-dust tests; local mocks.
T30	Check that touched state does not persist after the induced revert	PASS — same tested rollback path and fields only; not a universal invariant.

18.6.6 API and integration: T31–T36

ID	Scenario and setup	Result and evidence
T31	Valid sample request against local JSON policy fixtures	PASS, SIMULATED — six scenario outputs match; direct sample returns `ALLOW`.
T32	Policy API endpoint unavailable to an active client	NOT EXECUTED — no hosted policy API or configured production client.
T33	Malformed HTTP decision response reaches a fail-closed client	NOT EXECUTED — no active response-validation client is wired to mock.
T34	Mock request omits required `amount` field	PASS, SIMULATED, LIMITED — HTTP 422; not full OpenAPI validation.
T35	Client receives an incorrect or stale policy version	NOT EXECUTED — no supported live client path.
T36	Replay Registry update events and compare policy history	PASS, LIMITED — local event replay only; not hosted API audit-log consistency.

18.7 Reproduction commands

For the root contract suite, initialize submodules and use `forge fmt --check`, `forge build --sizes` and `forge test -vv`. For the rollback case, run `forge test --match-test test_V3_DustRefundRevertsWhenTheTokenReturnsFalse -vvv`. For Majlis, use `npm ci`, `npm run typecheck`, `npm test` and `npm run build` under `apps/majlis`. For SDK, use `npm ci`, `npm test` and `npm run build` under `gravitas-sdk`. For the mock API, run `npm ci` and `npm start` in `integration-kit/mock-server`, then `npm ci` and `GRAVITAS_API=http://127.0.0.1:8787 npm run verify` in `integration-kit/sdk-examples`. The reviewer guide contains the exact configuration, reset steps and evidence paths.

19 Glossary

AML/CFT – **Anti-money laundering and countering the financing of terrorism**: Controls not implemented by the reviewed Registry allow-list or local mock.

API – **Application Programming Interface**: Here, a fixture-backed mock HTTP endpoint and an OpenAPI draft, not a verified hosted production service.

Atomicity – **Single-transaction atomic execution**: Within one EVM transaction, a revert rolls back the state changes made by that transaction; it does not remove broader market, governance or legal risk.

CBAR – **Central Bank of the Republic of Azerbaijan**: The central bank named as the intended exploratory discussion counterpart. No endorsement or approval is claimed.

Default deny – **Default-deny policy posture**: An unlisted or unknown sample in the local mock is returned as DENY; this does not establish global token interception.

EIP-712 – **Ethereum typed structured data signing**: A typed-data signing format used by the V3 intent domain and operation parameters.

EVM – **Ethereum Virtual Machine**: The execution environment for the reviewed Solidity contracts.

Signed Intent – **Signed operation intent**: An owner-authorized set of V3 migration parameters bound to a signer, chain, verifying contract, nonce and deadline.

KYC – **Know-your-customer checks**: Identity verification not implemented by the reviewed system.

Majlis **– Majlis governance application**: An application-level workflow for proposals, deliberation, votes, quorum and lifecycle records; it does not submit Registry updates.

NFT **– Non-fungible token**: The V3 position token representing a liquidity position.

Policy Registry **– Gravitas Policy Registry**: An owner-controlled smart contract storing allow-list, pause, version and policy-hash state.

RegTech **– Regulatory technology**: Technology supporting compliance and control processes; no production institution integration is claimed.

RPC **– Remote Procedure Call**: The JSON-RPC interface used for read-only chain calls in this package.

Arbitrum Sepolia **– Arbitrum Sepolia test network**: The test network used for the read-only deployment snapshot, chain ID 421614.

SHA-256 **– Secure Hash Algorithm 256-bit**: A hash function used in Majlis parameter canonicalization; a hash is not a signature or approval.

SupTech **– Supervisory technology**: Technology supporting supervisory processes; relevance is a hypothesis, not an endorsement.

Teleport V2 **– Teleport V2 liquidity migration contract**: A same-chain V2-style liquidity operation using a caller-supplied factory and authorized router; not a bridge.

Teleport V3 **– Teleport V3 position migration contract**: A same-chain migration of a Uniswap V3-style position NFT to a newly minted position.

Testnet **– Public blockchain test network**: A network for testing, not production settlement or evidence of a production service.

20 References

[1] CBAR Regulatory Sandbox portal; the third-cycle timing is documented separately. [2] CBAR announcement for the third application cycle, published 25 June 2026. [3] CBAR Financial Sector Development Strategy 2024–2026. [4] CBAR SupTech Roadmap overview. [5] CBAR announcement of four innovative products for sandbox testing, including Islamic-finance pilots. [6] Arbitrum official chain information, including Arbitrum Sepolia chain parameters. [7] PublicNode Arbitrum Sepolia endpoint information; the report identifies it as a third-party provider. [8] Arbiscan Sepolia Registry address page with public source/ABI material. [9] CBAR Regulation on the application of the Regulatory Sandbox, Board Resolution 06/4, 31 January 2024. [10] Official e-Qanun text of the Regulatory Sandbox regulation. [11] Arbiscan Sepolia Teleport V3 address page. [12] Arbiscan Sepolia Teleport V2 address page. [13] Arbiscan Sepolia Timelock address page.

21 Five founder actions

First, obtain an Azerbaijani legal-entity and applicant-eligibility opinion for the specific activity. Second, identify a regulated local institution that could independently define a narrow use case and potential pilot role. Third, commission an independent security review and resolve ownership, signer, executor, emergency-control and recovery design. Fourth, complete real integration, API authentication, data protection, AML/CFT, operational resilience and user-safeguard work. Fifth, provide verified team, market, funding target, use-of-funds and milestone data before investor circulation; keep fundraising separate from regulatory outreach.